DueBot — Multi-Agent Architecture

Voice + Text VC Due-Diligence Agent, Built on One Shared Stack

v2: Orchestrator, Financial Agent, Calendar Agent, Voice Agent, Text Agent

---

This extends the original DueBot spec (financial due diligence over iMessage) into a full multi-agent system: a VC can now call **or** text the same number, and an Orchestrator Agent routes the conversation to specialized agents underneath. Everyone on the team builds against the exact same tech stack and shared Supabase schema defined below — no one improvises their own database or model choice.

One-line pitch (unchanged): *VCs can't get more hours in the day, but they can get a diligence analyst in their pocket that answers financial due-diligence questions instantly — by voice or by text, over channels they already live in.*

The Five Agents

Agent	Role	Job
Voice Agent	Channel	Answers phone calls, streams speech in/out via Twilio
Text Agent	Channel	Sends/receives iMessages via Blooio (or Sendblue)
Orchestrator Agent	Router / brain	Classifies intent, calls the right specialist agent(s), composes the reply
Financial Agent	Specialist	Computes burn multiple, Rule of 40, LTV:CAC, runway, red flags
Calendar Agent	Specialist	Checks/books follow-up diligence or reference calls

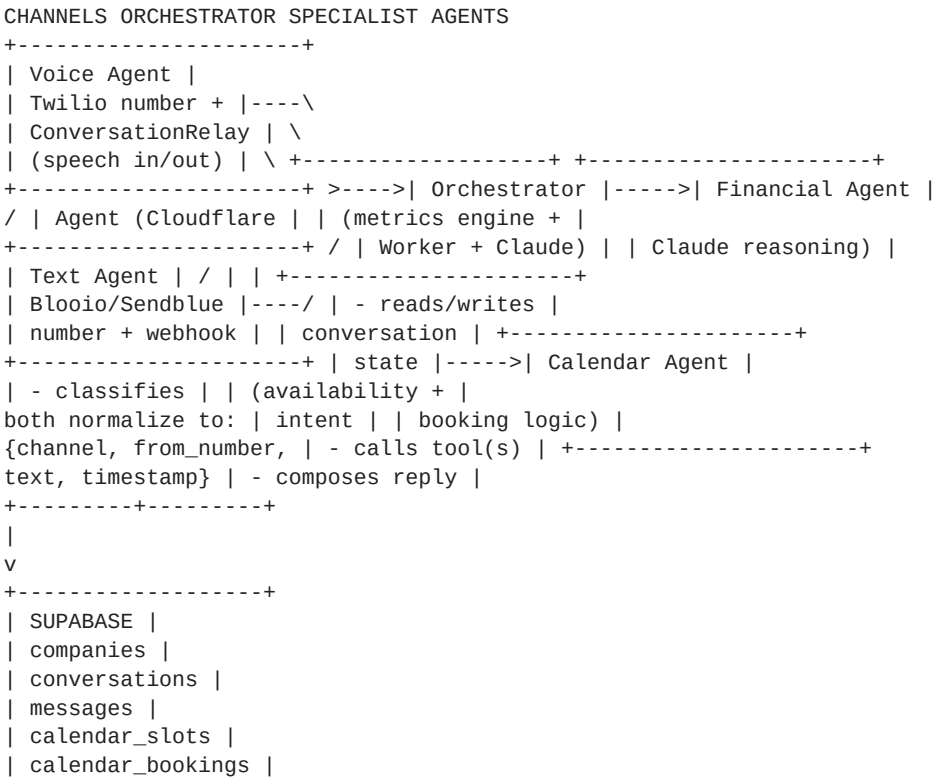
1. What We're Building (v2)

DueBot is now channel-agnostic. A VC can text the DueBot number and get an instant financial read on a portfolio/prospect company, then say “schedule a reference call with their CFO” and DueBot books it against a live (synthetic) calendar — or the VC can simply call the same number and continue the exact same conversation by voice, because identity is keyed on phone number in Supabase, shared across channels.

> **VC (text):** give me a burn multiple + rule of 40 check on Acme Robotics
>
> **DueBot:** Burn multiple 2.1x (benchmark <1.5x). Rule of 40: 34 (below threshold). Runway 11 months. ■■ CAC payback stretched to 19mo (was 13mo).
>
> **VC (text):** ok set up a reference call with their CFO
>
> **DueBot:** Booked: Priya Nair (CFO, Acme Robotics), Thursday 2:00–2:30pm. Confirmation sent.
>
> **VC (calls the same number later):** “hey, remind me what the customer concentration looked like for Acme?”
>
> **DueBot (by voice):** “Top 3 customers are 41% of ARR, and their largest single customer is 19%, renewing in 4 months with no multi-year contract on file.”

Problem stats recap: VCs spend **118 hrs avg. per deal** on diligence (184 hrs at late stage), deals take **83 days** to close, and diligence depth correlates with returns (**7.1x vs. 1.1x** for >40hrs vs. <20hrs). Full detail is in the companion problem-statement doc — this document is the build spec.

2. Multi-Agent Architecture



+------------------+

Design rule (unchanged): the Orchestrator and Financial Agent never do arithmetic inside the LLM. The LLM only picks tools and composes language; deterministic code computes every number. **Identity rule:** phone number is the single key that ties a person's conversation history together across voice and text — this is what makes “continue on a different channel” actually work.

Why this shape: one Orchestrator, two channel adapters, two specialist agents

- Channel adapters (Voice, Text) do the minimum needed to normalize input/output for their medium — they contain no business logic, so either channel can go down without breaking the others.
- The Orchestrator is the only place that holds conversation state and decides what to do — this keeps the “brain” in one place instead of duplicated logic across channels.
- Financial Agent and Calendar Agent are just callable functions/routes from the Orchestrator's point of view (tool-calling) — either can be built and tested completely independently of the channels.

3. Tech Stack — Lock This In as a Team Before Building

Everyone builds against this exact stack. Do not substitute a different database, model provider, or messaging API mid-build — the whole point of an orchestrator architecture is that the pieces only snap together if the contracts (Section 5) and stack are identical for all three people.

Layer	Choice	Why / Notes
Text channel	Blooio	REST + webhooks, free trial, no Apple hardware needed. Sendblue is the agreed fallback only.
Voice channel	Twilio Voice + ConversationRelay	Twilio number handles the call; ConversationRelay manages speech-to-text/text-to-speech over a WebSocket so you build against text, not raw audio.
Backend / compute	Cloudflare Workers	One shared Worker project. Routes: /voice, /text, /orchestrate, /agents/financial, /agents/calendar.
Database	Supabase (Postgres)	One shared project, one connection string, shared in team chat (never committed to the repo). RLS off for the hackathon — internal tool only.
LLM	Claude (Anthropic API)	Sonnet for orchestrator routing + financial reasoning (accuracy matters for numbers a VC will double-check). Gemini 2.5 Flash is the agreed zero-cost fallback if credits run out.
Repo	One shared GitHub repo	All three routes live in the same repo/deploy so the Orchestrator can call the other two as internal fetches without cross-service auth headaches.

First team action, before anyone writes agent logic: one person creates the Supabase project and the Cloudflare Workers project, and shares the connection string + API tokens with the other two immediately. This unblocks all three tracks in parallel.

4. Supabase Schema (Shared Contract — Freeze Early)

Table	Columns
companies	id, name, stage, sector, ask_amount, pre_money, arr, arr_growth_yoy, gross_margin, net_burn_monthly, net_new_arr_monthly, cash_on_hand, cac, ltv, cac_payback_months, cac_payback_months_prior, top3_pct_arr, largest_customer_pct_arr, largest_customer_renewal_months, multi_year_contracts, cohort_m1, cohort_m6, cohort_m12, arr_proj_12mo, arr_proj_18mo
conversations	id, phone_number (unique), channel_last_used, last_company_id, last_metrics_discussed, updated_at
messages	id, conversation_id (fk), channel, direction (in/out), content, created_at
calendar_slots	id, company_id (fk), contact_name, contact_role, slot_start, slot_end, is_booked
calendar_bookings	id, slot_id (fk), phone_number, purpose, created_at

conversations.phone_number is the identity key that lets a VC start on text and continue on voice. **calendar_slots** should be pre-seeded (not dynamically generated) with a handful of fake availability windows per

company contact for the next 5 business days — this avoids building real calendar OAuth under time pressure while still making the booking feel real in the demo.

5. Message & Tool-Call Contracts (Freeze Before Sync 1)

Channel → Orchestrator envelope (both Voice and Text adapters produce this exact shape):

```
{ "channel": "voice" | "text",
"from_number": "+1XXXXXXXXXX",
"text": "the message content, or transcribed speech",
"external_id": "call_sid or message_id",
"timestamp": "ISO 8601" }
```

Orchestrator → Financial Agent tool call:

```
{ "tool": "financial_agent",
"company_name": "Acme Robotics",
"requested_metrics": ["burn_multiple", "rule_of_40", "runway"] }
```

Orchestrator → Calendar Agent tool call:

```
{ "tool": "calendar_agent",
"action": "check_availability" | "book",
"company_name": "Acme Robotics",
"contact_role": "CFO" | "customer_reference",
"preferred_window": "this week" }
```

Whoever owns the Orchestrator (Part B below) publishes these three JSON shapes to the team in the first 30 minutes. Everyone else builds against the shape, not against each other's in-progress code.

6. Team Split — Three People, Five Agents

Mapped exactly to how you described it: one person owns the Financial Agent, one owns the Orchestrator (plus the lighter Calendar Agent alongside it, since the same person already owns routing/state), and one owns both channel agents (Voice + Text), since they share the same “get bytes in and out reliably” skillset.

Part	Owns	Core Scope
A	Financial Agent	Metrics engine, financial dataset, Supabase 'companies' table, financial reasoning prompt
B	Orchestrator + Calendar Agent	Routing/tool-calling brain, conversation state, 'conversations'/'messages'/'calendar_*' tables, repo/env setup
C	Voice Agent + Text Agent	Twilio + ConversationRelay, Blooio/Sendblue webhook, both channel adapters

PART A

Financial Agent

Owner: the finance/data-logic person

#	Task	Time
1	Set up the 'companies' table in Supabase per Section 4 schema.	0:00–0:30
2	Seed 4–6 fictional Series A/B/C companies; give one a clear, catchable red flag (e.g. stretched CAC payback or customer concentration).	0:30–1:15
3	Implement the metrics engine as pure functions behind /agents/financial: burn multiple, Rule of 40, LTV:CAC, runway, CAC payback, concentration flag. Unit test by hand against seed data.	1:15–2:15
4	Build the reply-composition prompt for financial answers: numbers-first, one flag, one suggested next question — analyst tone, not chatbot tone.	2:15–3:00
5	Expose /agents/financial as an internal route the Orchestrator can call with the Section 5 tool-call JSON, and return a clean JSON result back.	3:00–3:30
6	Integration pass with Part B once Orchestrator is routing real calls to it (Sync 2).	4:00–4:30
7	Edge cases: ambiguous company name, metric not in dataset, ties to red-flag phrasing for demo punch.	4:30–5:15

Formulas to implement exactly (never let the LLM compute these):

Metric	Formula	Read
Burn multiple	net burn ÷ net new ARR	<1.5x good; >2x flag
Rule of 40	YoY growth % + margin %	≥40 healthy
LTV:CAC	LTV ÷ CAC	>3x healthy
CAC payback (mo)	CAC ÷ (ARR/customer ÷ 12 × margin)	watch trend vs. prior period
Runway (mo)	cash on hand ÷ net burn monthly	flag if <12 months

Metric	Formula	Read
Concentration flag	top-3 % ARR >30% or top customer >15%	material risk Series B+

PART B

Orchestrator Agent + Calendar Agent

Owner: the routing/systems-logic person

#	Task	Time
1	Create the shared Cloudflare Workers project and Supabase project; distribute connection string/tokens to Parts A and C immediately.	0:00–0:20
2	Set up 'conversations' and 'messages' tables per Section 4; write read/write helpers keyed on phone_number.	0:20–1:00
3	Publish the three JSON contracts from Section 5 to the team.	0:30 (in parallel)
4	Build /orchestrate: accepts the channel envelope, loads conversation state, calls the LLM with financial_agent and calendar_agent as tool definitions, executes the chosen tool call(s).	1:00–2:30
5	Stand up a skeleton /orchestrate that returns a canned reply before the real logic is done, so Parts A and C can integrate against a live endpoint at Sync 1.	1:00–1:15
6	Build the Calendar Agent: 'calendar_slots'/'calendar_bookings' tables, seed fake availability for each company's CFO/customer contacts across the next 5 business days, check_availability + book logic behind /agents/calendar.	1:30–3:00
7	Wire real Financial Agent + Calendar Agent responses into /orchestrate, replacing the canned reply.	3:00–4:00 (Sync 2 target)
8	Reply composition: merge tool outputs into one natural final response, handle multi-turn follow-ups (“yeah, that one” referring to last company mentioned).	4:00–4:45
9	Resilience: timeouts on tool calls, graceful fallback reply, retry/idempotency on inbound webhooks.	4:45–5:30

Deliverables owned by Part B

- Live Supabase + Cloudflare projects, credentials distributed to the team in the first 20 minutes.
- The three frozen JSON contracts (Section 5), shared before Sync 1.
- /orchestrate route: intent classification, tool-calling, reply composition, conversation state.
- Calendar Agent with seeded fake availability and working booking logic.

PART C

Voice Agent + Text Agent

Owner: the channels/infra person

#	Task	Time
1	Sign up for Blooio (or Sendblue) trial; provision an iMessage-enabled number.	0:00–0:20
2	Build /text webhook: receive inbound iMessage, normalize to the Section 5 envelope, POST to /orchestrate, send the returned reply back via Blooio's send API.	0:20–1:00
3	Confirm a real round-trip text conversation on a phone against Part B's skeleton /orchestrate (canned reply) at Sync 1.	1:00–1:15
4	Set up a Twilio phone number and configure a TwiML/ConversationRelay endpoint for that number.	1:15–1:45
5	Build /voice: receive the ConversationRelay WebSocket session, forward transcribed speech as text into the same /orchestrate route, stream the reply back as speech.	1:45–3:00
6	Confirm a real phone call round-trip against the skeleton /orchestrate.	3:00–3:15
7	Once Part B wires real agent responses (Sync 2), re-test both channels end-to-end for correctness and acceptable latency.	4:00–4:45
8	Add resilience: graceful “one sec, let me check” filler on voice for LLM latency; webhook retry handling on text; WhatsApp/Twilio-SMS fallback plan if iMessage setup stalls.	4:45–5:30

Deliverables owned by Part C

- Working iMessage number wired end-to-end through /orchestrate.
- Working Twilio voice number wired end-to-end through /orchestrate via ConversationRelay.
- Both channels producing the identical Section 5 envelope, so Part B never special-cases the channel.
- Fallback plan (WhatsApp/SMS) ready but not needed unless iMessage setup stalls.

Biggest execution risk stays here: no official Apple iMessage API exists, and voice adds real-time latency constraints on top. Get bare echo versions of both channels working before minute 75, even before Part B's real logic exists — test against the canned-reply skeleton.

7. Combined Timeline (~9-hour build day)

Time	Part B	Part A	Part C
0:00–0:30	B: Supabase + Cloudflare projects created, credentials shared; contracts drafted	A: companies table + seed data	C: Blooio/Sendblue signup
0:30–1:15	B: conversations/messages tables + skeleton /orchestrate (canned reply)	A: seed data finished, metrics engine started	C: /text webhook built
— SYNC 1 (~1:15) —	Contracts frozen, skeleton /orchestrate live		
1:15–2:30	B: real /orchestrate logic + tool-calling LLM setup	A: metrics engine complete, reply prompt drafted	C: text round-trip confirmed; Twilio + ConversationRelay setup
2:30–4:00	B: Calendar Agent built + seeded availability	A: /agents/financial exposed and tested	C: /voice built, voice round-trip against skeleton
— SYNC 2 (~4:00) —	Real Financial + Calendar agents wired into /orchestrate		
4:00–5:30	B: reply composition, multi-turn handling, resilience	A: integration testing, edge cases, demo red-flag polish	C: end-to-end re-test both channels, latency/resilience
5:30–7:00	All: bug bash together, rehearse demo across both channels		
7:00–demo	All: final rehearsal as a team		

8. Definition of Done

- A real phone can text OR call the DueBot number and get a correct, numbers-first financial answer.
- A booking request (“schedule a call with their CFO”) results in a real row in calendar_bookings and a confirmation reply.
- A conversation started on text and continued on voice (or vice versa) keeps context, via phone_number.
- At least one company has a genuine, catchable red flag DueBot surfaces unprompted.
- Everyone can explain the stack the same way if a judge asks — one Supabase project, one Cloudflare project, one LLM provider, no ad hoc substitutions.
- A backup screen-recorded demo exists in case live network/voice issues hit in the room.